

Práctica 5B. Logging

Objetivo:

- Conocer y dominar, el uso de un sistema de log para el registro de sucesos relevantes del sistema durante su ejecución.

Conceptos y motivación del logging

El logging permite registrar los eventos relevantes que ocurren durante la ejecución de una aplicación. Aunque durante el aprendizaje de programación es habitual utilizar la salida de consola para obtener retroalimentación, esta opción presenta limitaciones importantes: la información no queda almacenada, no permite distinguir adecuadamente niveles de importancia, dificulta utilizar configuraciones diferentes según el entorno y puede saturar la salida estándar o interferir con otros procesos. Por ello, se establece como criterio que no se debe utilizar la salida por consola como mecanismo de registro de una aplicación, sino un sistema específico de logging.

Mediante logging se **generan** mensajes informativos sobre lo que sucede, se les proporciona un **formato** homogéneo, se **clasifican** según su importancia y se puede **explotar** posteriormente la información registrada para conocer qué ocurrió en una ejecución concreta. El log constituye, por tanto, un registro de eventos relevantes del sistema.

Información y niveles de logging

Los formatos, niveles y severidad siguen estándares definidos en RFC, como RFC 5424 (Syslog). Una entrada de log debe permitir identificar, como mínimo, el **nivel**, la **fecha** y el **mensaje**. El mensaje puede contener texto, valores de variables y, cuando procede, información asociada a una excepción.

Los niveles permiten priorizar mensajes y decidir mediante configuración cuáles se registran. En pruebas puede interesar un nivel de detalle alto, mientras que en producción normalmente se utiliza *INFO*; los mensajes *WARNING* y *ERROR* son especialmente relevantes. También pueden definirse niveles distintos para diferentes partes del código.

Nivel	Significado
<i>TRACE</i>	Detalle muy profundo y minucioso de la ejecución; permite monitorizar prácticamente todos los detalles.
<i>DEBUG</i>	Información de depuración, normalmente introducida por desarrolladores; aporta detalle sobre el proceso seguido.
<i>INFO</i>	Información general de la ejecución; suele mostrarse en producción y permite conocer acciones realizadas por el usuario.
<i>WARN</i>	Situación no crítica que permite continuar al sistema; no debe ocultar errores reales de ejecución.
<i>ERROR</i>	La acción realizada no puede completarse satisfactoriamente, aunque la aplicación puede continuar funcionando.
<i>FATAL</i>	Fallo grave que impide continuar la ejecución y normalmente provoca el cierre de la aplicación.

Qué guardar y dónde guardar los logs

Para facilitar la explotación de los registros es conveniente utilizar un patrón homogéneo en todas las líneas. Se recomienda registrar información como: la fecha, el identificador de sesión del usuario, el fichero de código que genera el log y el tiempo de ejecución del método, además del mensaje y la información que resulte útil.

```
20 nov 202 13:54:09,863 INFO MiCampusAuth:sessionCreated:126 - Usuario: pedrody@um.es -1700434809863 -
[96.249.89.158] - [Chrome-119.0.6045.123] - [Android 6.0.1 - Nexus 5-ARM]
```

Los registros pueden enviarse a la **salida estándar**, a **ficheros** o a **servicios externos**. El almacenamiento en ficheros o bases de datos permite mantener históricos y aplicar rotación. El *rotado del log* consiste en renombrar automáticamente el fichero que está recibiendo registros y comenzar a escribir en otro, evitando que un único fichero crezca indefinidamente.

Configuración en Java: Log4j2 y SLF4J

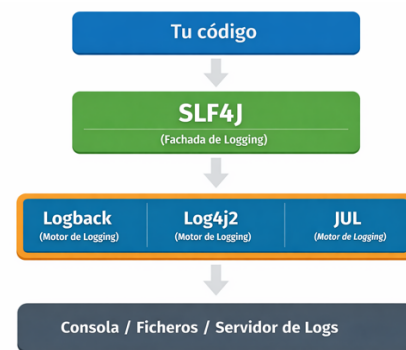
La configuración del logging en Java se plantea en dos pasos: (1) incorporar las librerías necesarias y (2) definir cómo se gestionará el registro. En el contexto de Maven se añaden las dependencias correspondientes.

Algunas librerías que podemos usar son:

1. *JUL* (Java Util Logging): JUL está incluido en la JDK, pero ofrece una configuración más tediosa y funcionalidad básica.
2. *Logback*: Ofrece buen rendimiento y configuración simplificada.

3. *Log4j2*. Destaca por su alto rendimiento, logging asíncrono nativo y elevada capacidad de configuración.

Nosotros usaremos *Log4j2* junto con *SLF4J* (*Simple Logging Facade for Java*). *SLF4J* actúa como fachada y abstrae la implementación concreta de logging, de modo que el código de la aplicación puede mantenerse independiente de la librería utilizada. Cambiar la implementación no obliga a modificar el código que utiliza el logger, aunque sí puede requerir cambiar los ficheros de configuración.



Pasos a seguir en la creación del proyecto:

1. Crear proyecto Java a partir de un arquetipo Maven.

New Maven Project

New Maven project
Select an Archetype

Catalog: Maven Central
Filter: maven-archetype-qu

Group Id	Artifact Id	Version
com.github.ywchang	maven-archetype-quickstart	1.1
com.haoxuer.maven.archetype	maven-archetype-quickstart	1.01
org.apache.maven.archetypes	maven-archetype-quickstart	1.5

Group Id: umu.tds
Artifact Id: PracticaLog
Version: 0.0.1-SNAPSHOT
Package: umu.tds.PracticaLog
☒ run archetype generation interactively

Properties available from archetype:

Name	Value
javaCompilerVersion	25
junitVersion	5.11.0

Ojo: Requiere tener instalado JDK 25

2. Añadir dependencias Maven.

Log4j2: <https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-core>

SLF4J: <https://mvnrepository.com/artifact/org.slf4j/slf4j-api>

Conectores *Log4J2* y *SLF4J*:

<https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-slf4j2-impl>

```

<!-- Dependencias log-->
<!-- Log4j2 -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.25.3</version>
</dependency>

<!-- SLF4J -->
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>2.0.17</version>
</dependency>

<!-- Conector log4j y slf4j -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-slf4j2-impl</artifactId>
  <version>2.25.3</version>
  <scope>compile</scope>
</dependency>
<!-- Fin dependencias log-->

```

3. Configurar log4j

Log4j2 requiere un fichero de configuración denominado obligatoriamente log4j2.xml, situado normalmente en src/main/resources. En él se define qué información se escribe, cómo se escribe y dónde se almacena.

La configuración se estructura principalmente mediante **Appenders** y **Loggers**.

Los **Appenders** son los componentes encargados de escribir los registros. Pueden ser de consola, fichero, base de datos, red, etc. (Console, File, Database, Network, ...), y es posible utilizar varios simultáneamente.

```

<Appenders>
  <!-- Appender de consola, se escribira por la consola -->
  <Console name="Console">
    <PatternLayout pattern="%d{HH:mm:ss} [%level] %logger - %msg%n" />
  </Console>

  <!-- Appender de fichero, esta configuracion es obligatoria para entornos de produccion -->
  <File name="FileLogger" fileName="logs/app.log" append="true">
    <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%level] %logger - %msg%n" />
  </File>
</Appenders>

```

El File Appender permite almacenar los registros en un fichero; su propiedad *fileName* determina la ruta y *append* indica si las nuevas líneas se añaden al final o sobrescriben el contenido.

```

<!-- Appender de fichero, esta configuracion es obligatoria para entornos de produccion -->
<File name="FileLogger" fileName="logs/app.log" append="true">
  <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%-5level] %logger - %msg%n" />
</File>

```

Cuando un fichero puede crecer indefinidamente se utiliza *RollingFile*. Este mecanismo aplica políticas de rotación y, mediante *filePattern*, determina el formato de los nombres de los ficheros generados durante la rotación.

```

<RollingFile name="RollingFile"
  fileName="logs/app.log"
  filePattern="logs/app-%d{yyyy-MM-dd}-%i.log.gz">
  <PatternLayout
    pattern="%d{yyyy-MM-dd HH:mm:ss} [%level] %logger - %msg%n" />
  <Policies>
    <!-- Rota cada 10 MB -->
    <SizeBasedTriggeringPolicy size="10 MB" />
    <!-- Rota cada día -->
    <TimeBasedTriggeringPolicy />
  </Policies>
</RollingFile>

```

PatternLayout define la información que aparece en cada línea. Entre los elementos utilizados se encuentran %d para fecha y hora, %level para el nivel, %logger para el nombre del logger, %msg para el mensaje y %n para el salto de línea.

```

<PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%level] %logger - %msg%n" />

```

Token	Descripción	Ejemplo
%d{yyyy-MM-dd HH:mm:ss}	Fecha y hora completa: año-mes-día <u>horas:minutos:segundos</u>	2026-01-08 14:45:30
% <u>level</u>	Nivel del log (INFO, DEBUG, WARN, ERROR...)	INFO, ERROR
%logger	Nombre del logger (normalmente la clase)	com.miapp.Main
% <u>msg</u>	Mensaje que envías en el código (log.info("..."))	Aplicación iniciada
%n	Salto de línea	Nueva línea al final

Los **Loggers** determinan qué *niveles* se registran, globalmente o por paquetes, y qué *Appenders* se utilizan. Se define un *logger Root* como configuración general y pueden *añadirse* *loggers* específicos para aplicar reglas diferentes a distintas partes del código. Así, por ejemplo, una configuración general puede registrar DEBUG o niveles superiores y permitir TRACE en un paquete concreto que necesite un mayor nivel de detalle.

```

<Loggers>
  <!-- Log general -->
  <!-- Damos de alta los appenders que queremos usar -->
  <Root level="DEBUG">
    <!-- Appender de consola -->
    <AppenderRef ref="Console" />
    <!-- Appender de fichero -->
    <AppenderRef ref="FileLogger" />
  </Root>

  <!-- Log sobre "umu.tds.trace" -->
  <Logger name="umu.tds.trace" level="TRACE" additivity="false">
    <!-- Appender de consola -->
    <AppenderRef ref="Console" />
  </Logger>
</Loggers>

```

En código Java, para obtener una instancia de log hacemos:

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory; } slf4j: Fachada para uso de log independiente de la implementación

public class LogDebugClass {
    private static final Logger log = LoggerFactory.getLogger(LogDebugClass.class);
    // Factoría de log

    public static void imprimeDebug() {
        log.debug("Imprimo mensaje de debug");
        log.trace("Imprimo trace desde el metodo de Debug");
        log.error("Error con codigo ", new Exception("Excepcion de error"));
    }
}
// Clase que usará esta instancia de log

// Uso de log

```

Ejemplos de Logging

Se deja en el repositorio un ejemplo resuelto en el que se abordan los siguientes requisitos:

Configuración del **log4j2** en un proyecto y definición de una estructura de clases que:

- Imprime log de nivel "INFO" para el paquete umu.pds.info
- Imprime log de nivel "DEBUG" para el paquete umu.tds.debug
- No imprime log de nivel "TRACE" en ningún caso.

Se invocan a los métodos que imprimen mensajes de debug de todos los niveles mencionados anteriormente independientemente de que el log lo imprima o no.

Además, dado que escribir mensajes de log concatenando valores es una mala práctica (`log.debug ("variable1: " + variable1 + "; variable 2: " + variable2);`), se plantea otro enfoque alternativo para especificar el valor de las variables sin hacer concatenaciones con el uso de la parametrización (ver <https://logging.apache.org/log4j/2.12.x/manual>)